

```

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <Adafruit_Fingerprint.h>
#include "LittleFS.h"
#include "RTCLib.h"
#include "BluetoothSerial.h"
#include <Preferences.h>
#include <ArduinoJson.h>

// --- CONFIGURACIÓN DE PINES
#define RXD2 16
#define TXD2 17
#define LED_VERDE 12
#define LED_ROJO 13

// --- CONFIGURACIÓN FIREBASE
const char* FB_BASE_URL =
"https://asistencia-93328-default-rtdb.firebaseio.com";
const char* FB_ASISTENCIA =
"https://asistencia-93328-default-rtdb.firebaseio.com/asistencia.json";

// --- OBJETOS
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&Serial2);
RTC_DS3231 rtc;
BluetoothSerial SerialBT;
WebServer server(80);
Preferences prefs;

// --- VARIABLES GLOBALES
String ssid, pass;
unsigned long lastSync = 0;
unsigned long btTimer = 0;
unsigned long lastWifiRetry = 0;
bool btActivo = true;
bool btAutenticado = false;
bool ventanaAbierta = true;
bool wifiIniciado = false;

// --- PROTOTIPOS
void activarModoWifi();
void verificarConexion();
void procesarWifiBT(String msg);
void procesarFechaBT(String msg);
void realizarResetTotal();
void syncTempToFirebase();
void verificarHuellaAsistencia();
void handleGetUsers();
void handleEnrol();
void handleDelete();
int findFreeID();
bool captureStep(int step);
void blinkLED(int pin, int veces);

```

```

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n[SISTEMA] --- INICIANDO SISTEMA INTEGRADO ---");

  // Pines de salida de LEDs configurados primero
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  digitalWrite(LED_VERDE, LOW);
  digitalWrite(LED_ROJO, LOW);

  // Apagar WiFi por completo al inicio para no saturar la antena
  WiFi.persistent(false);
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
  delay(200);

  // Inicialización de Almacenamiento y Reloj
  if (!LittleFS.begin(true)) Serial.println("[ERR] Fallo LittleFS");
  if (!rtc.begin()) {
    Serial.println("[ERR] Fallo RTC");
  } else {
    Serial.println("[OK] Módulo RTC DS3231 detectado.");
    mostrarFechaHoraRTC(); //
  }

  // Cargar credenciales guardadas
  prefs.begin("wifi-config", true);
  ssid = prefs.getString("ssid", "Vilers-2");
  pass = prefs.getString("pass", "Pabada686");
  prefs.end();

  // Inicialización del Lector de Huellas a 57600 baudios
  Serial2.begin(57600, SERIAL_8N1, RXD2, TXD2);
  finger.begin(57600);
  if (finger.verifyPassword()) {
    Serial.println("[OK] Sensor de huella en línea.");
  } else {
    Serial.println("[ERR] No se encontró el sensor de huella.");
  }

  // Encender Bluetooth en modo espera
  Serial.println("[BT] Modo Configuración Activado. Esperando comandos...");
  SerialBT.begin("Laggersoft");
  btTimer = millis();

  // RUTAS DEL SERVIDOR WEB (Peticiones desde la App/Web)
  server.serveStatic("/style.css", LittleFS, "/style.css");
  server.serveStatic("/script.js", LittleFS, "/script.js");

  server.on("/", []() {
    File file = LittleFS.open("/index.html", "r");
    if (!file) {

```

```

        Serial.println("[ERR] index.html no encontrado en LittleFS");
        server.send(404, "text/plain", "Falta index.html en memoria");
        return;
    }
    server.streamFile(file, "text/html");
    file.close();
});

server.on("/get_users", handleGetUsers);
server.on("/enrol", handleEnrol);
server.on("/delete", handleDelete);
}

void loop() {
    // 1. GESTIÓN COMUNICACIÓN BLUETOOTH
    if (btActivo) {
        if (ventanaAbierta || btAutenticado) {
            if (SerialBT.available()) {
                String msg = SerialBT.readStringUntil('\n');
                msg.trim();
                Serial.printf("[BT RX]: %s\n", msg.c_str());

                if (msg == "OBIWANKENOBI") {
                    btAutenticado = true;
                    ventanaAbierta = false;
                    SerialBT.println("Acceso OK.");
                } else if (msg == "CORTARBT" && btAutenticado) {
                    activarModoWifi();
                } else if (msg.startsWith("SSIDPASS:")) {
                    procesarWifiBT(msg);
                } else if (btAutenticado) {
                    if (msg.startsWith("DATETIME:")) procesarFechaBT(msg);
                    if (msg == "RESET") realizarResetTotal();
                }
            }
        }
    }
    // Si pasan 20 segundos sin autenticación, conmuta a WiFi automáticamente
    if (ventanaAbierta && (millis() - btTimer > 20000)) {
        Serial.println("[BT] Tiempo de espera agotado. Iniciando WiFi...");
        activarModoWifi();
    }
}

// 2. GESTIÓN COMUNICACIÓN WIFI Y PETICIONES WEB
if (wifiIniciado) {
    verificarConexion();
    server.handleClient();

    // Sincronizar fichadas con Firebase cada 30 segundos si hay internet
    if (WiFi.status() == WL_CONNECTED && (millis() - lastSync > 30000)) {
        syncTempToFirebase();
        lastSync = millis();
    }
}
}

```

```

// 3. GESTIÓN CONSTANTE DEL SENSOR DE HUELLAS
verificarHuellaAsistencia();
}

void activarModoWifi() {
    Serial.println("[RADIO] Apagando Bluetooth de forma segura...");
    SerialBT.end();
    btActivo = false;

    Serial.printf("[RADIO] Encendiendo WiFi. Conectando a: %s\n", ssid.c_str());
    WiFi.mode(WIFI_STA);
    delay(100);
    WiFi.begin(ssid.c_str(), pass.c_str());

    lastWifiRetry = millis() + 10000;
    server.begin();
    wifiIniciado = true;
}

void verificarConexion() {
    if (WiFi.status() != WL_CONNECTED) {
        if (millis() - lastWifiRetry > 20000) {
            lastWifiRetry = millis();
            Serial.println("[WIFI] Buscando red, reintentando conexión...");
            WiFi.begin(ssid.c_str(), pass.c_str());
        }
    } else {
        static bool logConexion = false;
        if (!logConexion) {
            Serial.print("[WIFI] ¡Conectado con éxito! Dirección IP: ");
            Serial.println(WiFi.localIP());
            logConexion = true;
        }
    }
}

void procesarWifiBT(String msg) {
    int c1 = msg.indexOf('['), c2 = msg.indexOf(']');
    int c3 = msg.lastIndexOf('['), c4 = msg.lastIndexOf(']');
    if (c1 != -1 && c3 != -1) {
        ssid = msg.substring(c1 + 1, c2);
        pass = msg.substring(c3 + 1, c4);

        prefs.begin("wifi-config", false);
        prefs.putString("ssid", ssid);
        prefs.putString("pass", pass);
        prefs.end();

        Serial.println("[OK] Nuevos datos de WiFi guardados. Reiniciando placa...");
        delay(500);
        ESP.restart();
    }
}

```

```

void procesarFechaBT(String msg) {
    int start = msg.indexOf('[') + 1;
    int end = msg.lastIndexOf(']');
    String valStr = msg.substring(start, end);
    int v[6];
    int count = 0;
    int lastPos = 0;
    for (int i = 0; i <= valStr.length(); i++) {
        if (i == valStr.length() || valStr[i] == ',') {
            v[count++] = valStr.substring(lastPos, i).toInt();
            lastPos = i + 1;
        }
        if (count == 6) break;
    }
    if (count == 6) {
        rtc.adjust(DateTime(v[0], v[1], v[2], v[3], v[4], v[5]));
        Serial.println("[OK] Hora del módulo RTC sincronizada.");
    }
}

void realizarResetTotal() {
    Serial.println("[SISTEMA] Borrando datos de almacenamiento y memoria...");
    finger.emptyDatabase();
    LittleFS.remove("/fichadas.csv");
    LittleFS.remove("/temp.csv");
    prefs.begin("wifi-config", false);
    prefs.clear();
    prefs.end();
    delay(500);
    ESP.restart();
}

void verificarHuellaAsistencia() {
    if (finger.getImage() == FINGERPRINT_OK) {
        if (finger.image2Tz() == FINGERPRINT_OK && finger.fingerFastSearch() ==
FINGERPRINT_OK) {
            DateTime now = rtc.now();
            String data = String(finger.fingerID) + "," + now.timestamp();
            Serial.printf("[ASISTENCIA] ID de Huella %d detectado correctamente.\n",
finger.fingerID);

            File f1 = LittleFS.open("/fichadas.csv", FILE_APPEND);
            if(f1) { f1.println(data); f1.close(); }

            File f2 = LittleFS.open("/temp.csv", FILE_APPEND);
            if(f2) { f2.println(data); f2.close(); }

            blinkLED(LED_VERDE, 1);
        } else {
            Serial.println("[ASISTENCIA] Advertencia: Huella dactilar no
registrada.");
            blinkLED(LED_ROJO, 1);
        }
    }
}

```

```

    delay(1000); // Evita lecturas duplicadas continuas
}
}

```

```

void handleGetUsers() {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String url = String(FB_BASE_URL) + "/tbl_alumnos.json";

    Serial.printf("[WEB SERVER] GET Alumnos -> %s\n", url.c_str());
    http.begin(client, url);
    int code = http.GET();

    Serial.printf("[WEB SERVER] Firebase respondió Código HTTP: %d\n", code);
    server.send(code, "application/json", http.getString());
    http.end();
}

```

```

void handleEnrol() {
    server.setHeader("Access-Control-Allow-Origin", "*");
    String uid = server.arg("uid");
    int id = findFreeID();
    if (id == -1) {
        server.send(200, "text/plain", "No hay espacio para mas huellas");
        return;
    }

    if (!captureStep(1)) {
        server.send(200, "text/plain", "Error: No se detecto el dedo");
        return;
    }
    blinkLED(LED_VERDE, 1);
    Serial.println("[ENROL] Paso 1 Completado. Retire el dedo...");

    unsigned long waitTime = millis();
    while (finger.getImage() != FINGERPRINT_NOFINGER) {
        if (millis() - waitTime > 5000) break;
        delay(100);
    }
    delay(100);
    Serial.println("[ENROL] Coloque el mismo dedo nuevamente...");
    delay(1000);

    if (captureStep(2) && finger.createModel() == FINGERPRINT_OK &&
    finger.storeModel(id) == FINGERPRINT_OK) {
        WiFiClientSecure client;
        client.setInsecure();
        HTTPClient http;
        String url = String(FB_BASE_URL) + "/tbl_alumnos/" + uid + "/huellaId.json";

        http.begin(client, url);
        int httpResponseCode = http.PUT(String(id));
    }
}

```

```

    Serial.printf("[ENROL] Actualizando Firebase. Código HTTP: %d\n",
httpResponseCode);

    if (httpResponseCode == 200) {
        server.send(200, "text/plain", "OK");
        blinkLED(LED_VERDE, 3);
        Serial.printf("[ENROL] Éxito. Huella asignada al ID de Sensor: %d\n", id);
    } else {
        server.send(200, "text/plain", "Error Firebase");
        blinkLED(LED_ROJO, 3);
    }
    http.end();
} else {
    server.send(200, "text/plain", "Fallo: Las huellas no coinciden");
    blinkLED(LED_ROJO, 2);
}
}

void handleDelete() {
    server.setHeader("Access-Control-Allow-Origin", "*");
    String uid = server.arg("uid");
    if (uid == "") {
        server.send(400, "text/plain", "Falta UID");
        return;
    }

    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;

    String urlUser = String(FB_BASE_URL) + "/tbl_alumnos/" + uid + ".json";
    http.begin(client, urlUser);
    int httpCode = http.GET();

    if (httpCode == 200) {
        String payload = http.getString();
        JsonDocument doc; // Compatible con ArduinoJson v6 y v7 sin desbordamiento
        deserializeJson(doc, payload);
        int idSensor = doc["huellaId"];

        if (idSensor > 0 && idSensor <= 1000) {
            if (finger.deleteModel(idSensor) == FINGERPRINT_OK) {
                Serial.printf("[DELETE] Removido ID %d de la memoria física del
sensor\n", idSensor);
            }
        }
        http.end(); // Libera la petición GET antes de iniciar la solicitud DELETE

        // Petición DELETE a Firebase
        http.begin(client, urlUser);
        int deleteCode = http.sendRequest("DELETE");
        Serial.printf("[DELETE] Borrando en Firebase. Código HTTP: %d\n",
deleteCode);
    }
}

```

```

    if (deleteCode == 200 || deleteCode == 204) {
        server.send(200, "text/plain", "OK");
        blinkLED(LED_VERDE, 2);
        Serial.println("[DELETE] Alumno purgado del ecosistema.");
    } else {
        server.send(500, "text/plain", "Error al eliminar de Firebase");
    }
    http.end();
} else {
    server.send(404, "text/plain", "Usuario no encontrado");
    http.end();
}
}

bool captureStep(int step) {
    int p = -1;
    unsigned long timeout = millis();
    while (p != FINGERPRINT_OK && (millis() - timeout < 2500)) {
        p = finger.getImage();
    }
    return (p == FINGERPRINT_OK && finger.image2Tz(step) == FINGERPRINT_OK);
}

int findFreeID() {
    for (int i = 1; i <= 1000; i++) {
        if (finger.loadModel(i) != FINGERPRINT_OK) return i;
    }
    return -1;
}

void blinkLED(int pin, int veces) {
    for (int i = 0; i < veces; i++) {
        digitalWrite(pin, HIGH);
        delay(200);
        digitalWrite(pin, LOW);
        delay(200);
    }
}

void mostrarFechaHoraRTC() {
    if (!rtc.begin()) {
        Serial.println("[RTC] Error: El módulo DS3231 no está respondiendo en el bus I2C.");
        return;
    }

    // Obtener la fecha y hora actual del chip
    DateTime ahora = rtc.now();

    Serial.println("\n--- [DIAGNÓSTICO RTC] ---");
    // Imprimir usando el formato estándar ISO (AAAA-MM-DDTHH:MM:SS)
    Serial.printf("Fecha/Hora Actual (Timestamp): %s\n",
        ahora.timestamp().c_str());

    // Imprimir desglosado por si necesitas verificar valores individuales

```



```

Serial.printf("Detalle: %02d/%02d/%04d %02d:%02d:%02d\n",
              ahora.day(), ahora.month(), ahora.year(),
              ahora.hour(), ahora.minute(), ahora.second());

// Verificar si el RTC perdió la hora (por ejemplo, si se quedó sin pila)
if (ahora.year() < 2020) {
    Serial.println("[ALERTA RTC] La fecha es muy antigua. Es probable que la
batería CR2032 esté agotada o falte sincronización.");
}
Serial.println("-----\n");
}

void syncTempToFirebase() {
    if (!LittleFS.exists("/temp.csv")) return;
    File f = LittleFS.open("/temp.csv", "r");
    if (!f) return;

    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    bool todook = true;
    Serial.println("[FIREBASE] Sincronizando registros con formato
Diccionario...");

    while (f.available()) {
        String line = f.readStringUntil('\n');
        line.trim();
        if (line.length() > 0) {
            // 1. Generamos un ID único variable numérico para el nodo asistencias
            // Usamos el tiempo en milisegundos sumado a un número aleatorio para
simular el ID largo
            unsigned long idVariable = 1770000000000ULL + (millis() % 9192516961ULL);

            // 2. Construimos la URL apuntando directamente al ID dinámico creado
            String urlDestino = String(FB_BASE_URL) + "/asistencia/" +
String(idVariable) + ".json";

            http.begin(client, urlDestino);
            http.addHeader("Content-Type", "application/json");

            // 3. Formateamos el payload exacto: "{\"fichada\":\"VALOR\"}"
            // Escapamos las comillas internas para que Firebase lo guarde como String
literal de JSON
            String jsonPayload = "{\"\\"fichada\\":\\" + line + "\\\"}";

            // Enviamos con PUT para escribir directamente en la ubicación exacta del
ID
            int code = http.PUT(jsonPayload);

            if (code == 200 || code == 201) {
                Serial.printf("[FIREBASE] Enviado a ID %s: %s (Código %d)\n",
String(idVariable).c_str(), line.c_str(), code);
            } else {

```

```

        Serial.printf("[ERR] Falló envío de ID %s. HTTP Código: %d\n",
String(idVariable).c_str(), code);
        todook = false;
    }
    http.end();
    delay(100); // Pequeña pausa para no saturar la conexión
}
}
f.close();

if (todook) {
    LittleFS.remove("/temp.csv");
    Serial.println("[FIREBASE] Sincronización finalizada. Archivo temporal
limpio.");
} else {
    Serial.println("[FIREBASE] Operación parcial. Se reintentará en el próximo
ciclo.");
}
}
}

```

```

// funcion anterior de guarda formato de fichada que no corresponde

// void syncTempToFirebase() {
//     if (!LittleFS.exists("/temp.csv")) return;
//     File f = LittleFS.open("/temp.csv", "r");
//     if (!f) return;

//     WiFiClientSecure client;
//     client.setInsecure();
//     HTTPClient http;
//     bool todook = true;

//     Serial.println("[FIREBASE] Sincronizando registros pendientes...");

//     while (f.available()) {
//         String line = f.readStringUntil('\n');
//         line.trim();
//         if (line.length() > 0) {
//             http.begin(client, FB_ASISTENCIA);
//             http.addHeader("Content-Type", "application/json");

//             String jsonPayload = "{\"fichada\":\"" + line + "\"}";
//             int code = http.POST(jsonPayload);

//             if (code == 200 || code == 201) {
//                 Serial.printf("[FIREBASE] Fichada enviada: %s (Código %d)\n",
line.c_str(), code);
//             } else {
//                 Serial.printf("[ERR] Falló envío de %s. HTTP Código: %d\n",
line.c_str(), code);

```

```
//      tolook = false;
//    }
//    http.end();
//  }
// }
// f.close();

// if (todook) {
//   LittleFS.remove("/temp.csv");
//   Serial.println("[FIREBASE] Sincronización finalizada. Archivo temporal
limpio.");
// } else {
//   Serial.println("[FIREBASE] Operación parcial. Se reintentará en el
próximo ciclo.");
// }
// }
```